



Liberté • Égalité • Fraternité
RÉPUBLIQUE FRA



Rapport de Projet Drone cible

Partie personnelle
BTS CIEL – option IR

JAUMOUILLE Gatien
2025-2026

1. Périmètre d'activité	3
1.1 Mon rôle dans le projet	3
1.2 Liste de mes activités individuelles	3
1.3 Diagramme de cas d'utilisation de mon périmètre	3
2. Environnement de développement	4
2.1 Justification des outils et langages choisis	4
2.2 Justification des librairies utilisées	5
2.3 Identification des contraintes du client	6
3. Architecture du projet	7
3.1 Diagramme d'architecture logicielle et matérielle	7
4. Développement – état d'avancement	8
4.1 Présentation des composants développés	8
4.2 Extrait de code commentés et expliqués	9
4.3 Tableau ou cahier de tests unitaires	13
4.4 Gestion des versions	15
5. Exploitation réseau et matériels	15
5.1 Présentation des équipements	15
5.2 Justification des choix matériels	16
5.3 Protocoles et formats de données échangés	16
5.4 Résultats d'outils d'analyse réseau	17
5.5 Gestion des versions	18
5.6 Cybersécurité	19
6. Bilan personnel	19
6.1 Ce que J'ai réalisé	19
6.2 Difficultés rencontrées et solutions apportées	19
6.3 Ce que j'ai appris	20
Annexes	21
[1] Versionnement git	21
[2] Code complets du noeud imu_node : imu_data.py	22
[3] Code complet du noeud node_consigne : node_consigne.py	26
[4] Fichier waypoints en GEOJson : waypoints.json	31
[5] Capture d'écran d'une trame du capteur FXAS21002 au PicoScope	34

1. Périmètre d'activité

1.1 Mon rôle dans le projet

Mon rôle dans ce projet a été de développer et mettre en œuvre le mode de pilotage autonome du drone cible. Ce mode permet au drone de se déplacer de façon entièrement automatique en suivant une série de points de passage (waypoints) définis dans un fichier prédéfini. J'ai été en charge de l'intégration des capteurs IMU et GPS dans l'architecture ROS2-Jazzy, du traitement et de la fusion de leurs données pour estimer la position et l'orientation du drone, ainsi que du développement des nœuds permettant le calcul et l'exécution des commandes de navigation autonome. J'ai également assuré la mise en place du conteneur Docker associé au mode autonome.

1.2 Liste de mes activités individuelles

Dans le cadre de ce projet, mes activités individuelles ont été les suivantes :

- Développement du nœud `imu_node` : lecture des données des capteurs FXOS8700 (accéléromètre et magnétomètre) et FXAS21002 (gyroscope) via le bus I2C, et publication des données de rotation et de boussole sur les topics ROS2 `/imu/rotation` et `/imu/boussole`.
- Développement du nœud `node_consigne` : récupération des waypoints issus du fichier prédéfini et publication sur le topic `/consigne_waypoint`, ainsi que transmission des points de passage au nœud de navigation.
- Intégration du driver NMEA NavSat Driver : configuration du driver ROS2 permettant de lire et d'interpréter les trames NMEA provenant du module GPS via liaison USB/UART, et publication des données de position sur le topic `/fix`.
- Intégration du nœud Robot Localization : configuration de la fusion des données GPS et IMU pour produire une odométrie fiable publiée sur le topic `/odom_filter`.
- Développement du nœud `robot_controller` : abonnement au topic `/cmd_vel` et pilotage des actionneurs physiques via les interfaces GPIO (broches 17, 18, 27, 22) pour les moteurs via le driver L298N, et via I2C pour le servomoteur via le module PCA9685.
- Intégration Docker : mise en place du conteneur ROS2-Jazzy nécessaire au fonctionnement du mode autonome sur la Raspberry Pi 5, incluant l'installation de toutes les dépendances système et bibliothèques Python.
- Tests et validation : vérification de la détection des capteurs sur le bus I2C, validation de la réception GPS et tests de déplacement autonome sur waypoints.

Le développement du nœud `node_navigation_auto` est la seule activité restante à réaliser dans le cadre de mon périmètre.

- Développement du nœud `node_navigation_auto` : ce nœud aura pour rôle d'exploiter les données de localisation filtrées publiées sur le topic `/odom_filter` ainsi que les waypoints reçus sur `/consigne_waypoint`, afin de calculer les commandes de déplacement et de les publier sur le topic `/cmd_vel`.

1.3 Diagramme de cas d'utilisation de mon périmètre

Le diagramme de cas d'utilisation ci-dessous délimite le périmètre fonctionnel des composants dont j'avais la charge, en identifiant les acteurs principaux : le fichier de waypoints qui définit la

mission, le module GPS qui assure la localisation géographique, le capteur IMU (FXAS21002 + FXOS8700) qui fournit les données inertielles, et les moteurs/servo-moteur qui constituent les actionneurs physiques du drone.

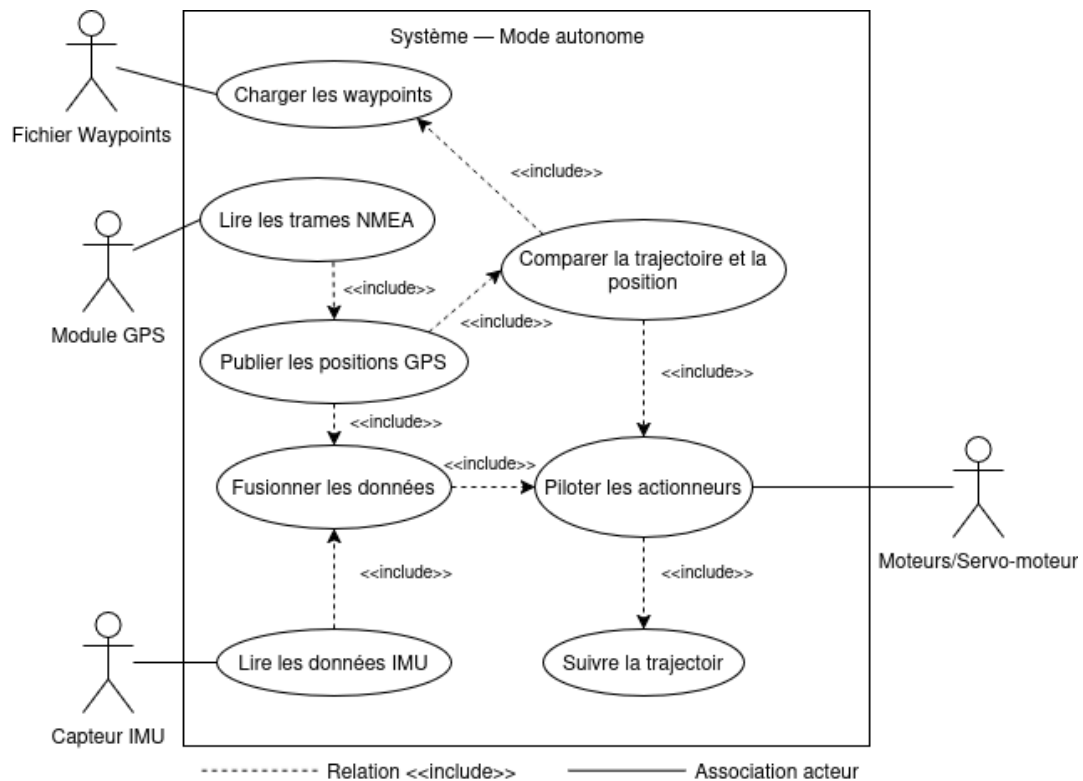


Diagramme d'architecture logicielle et matérielle du mode autonome

Ce diagramme illustre les interactions entre les acteurs externes et le système de navigation autonome. Le module GPS transmet ses trames NMEA au cas d'utilisation "Lire les trames NMEA", qui via une relation «include» alimente la publication des positions GPS. Ces positions sont ensuite fusionnées avec les données du capteur IMU par le cas d'utilisation "Fusionner les données", qui reçoit également les consignes du fichier de waypoints via "Charger les waypoints".

Les données fusionnées permettent d'alimenter le cas d'utilisation "Comparer la trajectoire et la position", qui constitue le cœur de la logique de navigation autonome. Il reçoit en «include» les waypoints chargés ainsi que les positions GPS publiées, et transmet ensuite les instructions au cas d'utilisation "Piloter les actionneurs". Ce dernier commande physiquement les moteurs et le servo-moteur du drone, et inclut également le cas d'utilisation "Suivre la trajectoire", qui assure le suivi continu du drone le long des points de passage définis.

2. Environnement de développement

2.1 Justification des outils et langages choisis

Dans le cadre du projet Drone Cible, le développement logiciel a été réalisé principalement en Python. Ce langage a été choisi pour plusieurs raisons. Tout d'abord, Python est un langage simple à utiliser et très lisible, ce qui facilite la maintenance du code et la collaboration entre les différents membres de l'équipe. De plus, Python possède de nombreuses bibliothèques permettant d'interagir facilement avec des capteurs et des interfaces matérielles.

Dans la partie du mode autonome du projet Drone Cible, le système repose sur l'utilisation de Docker et ROS2, comme détaillé dans le rapport commun. Plusieurs nœuds sont utilisés pour :

- la lecture des données du capteur IMU
- la lecture des données GPS
- la gestion de la navigation

Afin d'assurer le bon fonctionnement de ces différents nœuds, plusieurs bibliothèques Python ont été utilisées. Le choix de ces bibliothèques a été guidé par les besoins spécifiques de chaque composant du système.

2.2 Justification des librairies utilisées

Pour faciliter le développement du système du drone cible, plusieurs bibliothèques Python et dépendances système ont été utilisées. Ces librairies permettent d'assurer la communication avec les capteurs, le traitement des données et l'intégration avec l'environnement ROS2.

Dans l'environnement Docker utilisé pour le mode autonome, plusieurs dépendances ont été installées afin de garantir le bon fonctionnement des différents modules du projet. L'image de base utilisée est `ros:jazzy-ros-base`, ce qui intègre directement ROS2-Jazzy dans le conteneur.

Parmi ces dépendances, on retrouve notamment :

- `python3-pip` : permet l'installation et la gestion des bibliothèques Python.
- `python3-venv` : permet de créer un environnement virtuel Python isolé afin d'éviter les conflits entre les bibliothèques installées.
- `python3-dev` et `build-essential` : outils de compilation nécessaires pour installer certaines bibliothèques Python nécessitant une compilation native.
- `python3-colcon-common-extensions` : outil utilisé pour compiler et gérer le workspace ROS2.
- `ros-jazzy-rmw-cyclonedds-cpp` : middleware utilisé par ROS2 pour la communication entre les différents nœuds du système.
- `ros-jazzy-nmea-navsat-driver` : driver ROS2 permettant de lire et d'interpréter les trames NMEA provenant du module GPS.
- `ros-jazzy-robot-localization` : package ROS2 fournissant des algorithmes de fusion de données (GPS, IMU) pour estimer la position du drone.
- `gpsd` et `gpsd-clients` : outils permettant de gérer et d'accéder aux données provenant du module GPS.
- `python3-gps` : bibliothèque permettant d'interagir avec le service `gpsd`.
- `iproute2` et `iputils-ping` : outils réseau permettant de tester la connectivité et de diagnostiquer les communications réseau.
- `nano` : éditeur de texte utilisé pour modifier les fichiers de configuration directement dans le conteneur.

Certaines bibliothèques Python spécifiques ont également été installées via `pip` afin d'interagir avec les capteurs utilisés dans le projet, notamment `flask` pour une éventuelle interface web et `smbus2` pour la communication I2C.

Le capteur IMU (Inertial Measurement Unit) permet de mesurer différents paramètres physiques du drone tels que l'accélération, la rotation et le champ magnétique. Ces données sont utilisées pour estimer l'orientation et les mouvements du drone cible.

Les bibliothèques suivantes sont utilisées pour la gestion de ce capteur :

- `rcipy` : bibliothèque Python permettant d'interagir avec ROS2. Elle permet de créer des nœuds ROS2 et de gérer la communication entre les différents composants du système.
- `sensor_msgs.msg.Imu` : message standard fourni par ROS2 permettant de transmettre les données provenant des capteurs inertiels.
- `board` et `busio` : bibliothèques de la suite Adafruit CircuitPython permettant d'initialiser et de gérer la communication sur le bus I2C du Raspberry Pi.
- `adafruit_fxos8700` : bibliothèque Adafruit dédiée au capteur FXOS8700, permettant de lire les données d'accélération et de champ magnétique.
- `adafruit_fxas21002c` : bibliothèque Adafruit dédiée au capteur FXAS21002, permettant de lire les données de rotation (gyroscope 3 axes).
- `math` : utilisée pour effectuer les calculs mathématiques nécessaires au traitement des données des capteurs.

Ces bibliothèques permettent de récupérer les données des capteurs IMU et de les publier dans le réseau ROS2 afin qu'elles puissent être utilisées par les autres composants du système, notamment le nœud Robot Localization.

Le drone cible utilise également un module GPS afin de déterminer sa position géographique. Ces informations sont utilisées pour suivre un itinéraire composé de plusieurs points de passage appelés waypoints.

Les bibliothèques suivantes sont utilisées pour gérer les données GPS :

- `rcipy` : permet d'intégrer le module GPS dans l'architecture ROS2 sous forme de nœud.
- `geometry_msgs.msg.Point` : message ROS2 permettant de représenter une position géographique dans l'espace.
- `sensor_msgs.msg.NavSatFix` : message ROS2 standard utilisé pour transmettre les données de position GPS (latitude, longitude, altitude) entre les nœuds du système.
- `json` : bibliothèque utilisée pour lire les fichiers de configuration contenant les coordonnées des waypoints.
- `math` : utilisée pour effectuer les calculs géométriques nécessaires, tels que le calcul de distance et de cap entre deux points GPS.

Grâce à ces différentes bibliothèques, il est possible de récupérer les coordonnées GPS du drone cible, de traiter les données et de les transmettre aux autres nœuds du système via ROS2, notamment au nœud `node_navigation` qui calcule la distance entre le drone et le waypoints.

2.3 Identification des contraintes du client

Le mode autonome doit respecter plusieurs contraintes définies dans le cahier des charges.

Le système de navigation doit être capable de suivre un itinéraire avec une précision d'environ ± 5 mètres. Cette contrainte impose une utilisation fiable du module GPS ainsi qu'un traitement correct des données de position.

Le système doit permettre de définir facilement un itinéraire pour le drone. Pour cela, les points de navigation sont stockés dans un fichier GeoJSON avec les coordonnées en latitude et en longitude.

Cette solution permet de modifier facilement le trajet du drone sans avoir à modifier directement le programme.

Plusieurs dispositifs de sécurité doivent être intégrés au système afin d'éviter un comportement dangereux du drone.

Tout d'abord, si le signal GPS ou la communication est perdue pendant une durée supérieure à 3 secondes, le mode autonome est automatiquement désactivé.

De plus, un bouton d'arrêt d'urgence est prévu afin de pouvoir arrêter immédiatement le fonctionnement du drone en cas de problème.

3. Architecture du projet

3.1 Diagramme d'architecture logicielle et matérielle

Voici ci-dessous le diagramme d'architecture logicielle et matérielle du mode autonome :

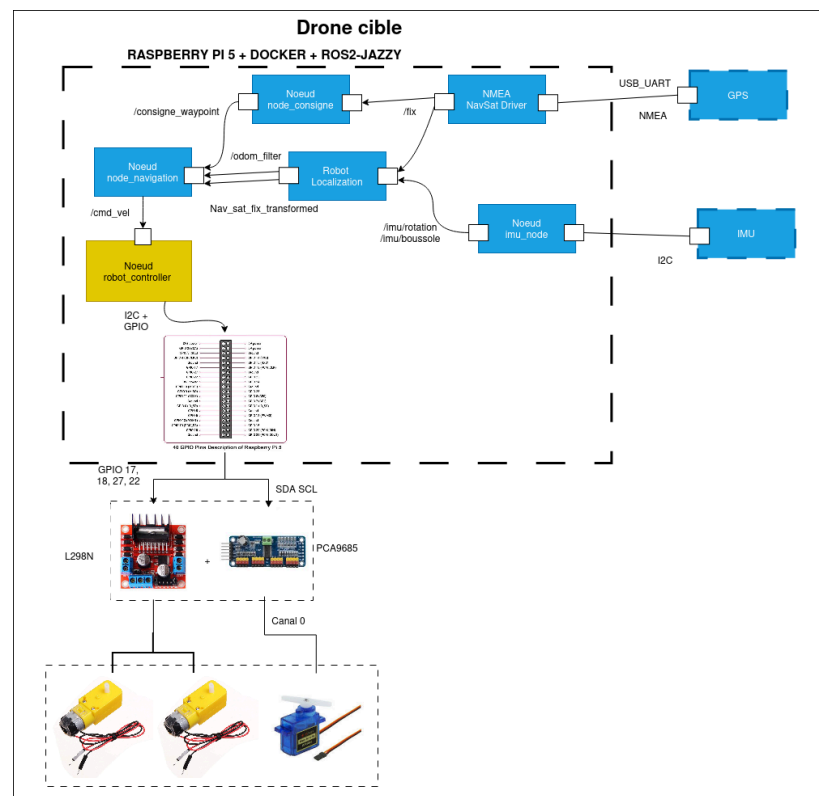


Diagramme d'architecture logicielle et matérielle du mode autonome

Ce diagramme présente l'architecture du drone cible en mode autonome. Le système est basé sur un Raspberry Pi 5 exécutant ROS2-Jazzy dans un conteneur Docker. Les différents programmes sont organisés sous forme de nœuds ROS2 qui communiquent entre eux via des topics.

Le nœud `node_consigne` récupère les données de position GPS via le driver NMEA NavSat Driver, qui communique avec le module GPS par liaison USB_UART en protocole NMEA. Il publie les waypoints sur le topic `/consigne_waypoint`. Le nœud `imu_node` lit quant à lui les

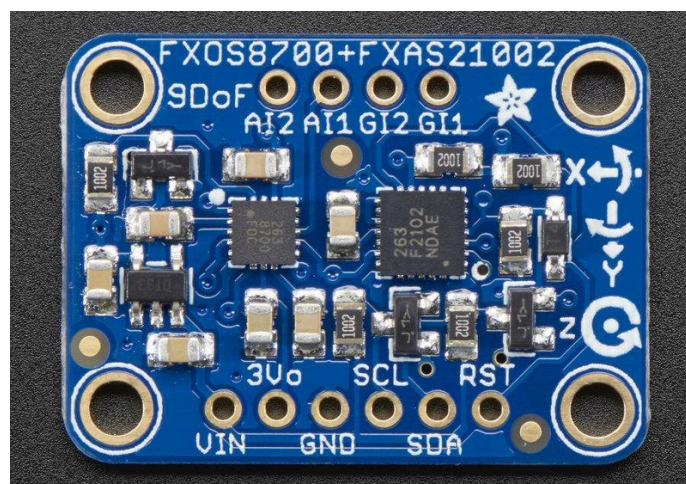
données des capteurs IMU (FXAS21002 et FXOS8700) via le bus I2C, et publie les données de rotation et de boussole sur les topics /imu/rotation et /imu/boussole.

Ces informations sont fusionnées par le nœud Robot Localization, qui combine les données GPS (/fix, Nav_sat_fix_transformed) et IMU pour produire une odométrie filtrée publiée sur le topic /odom_filter. Le nœud node_navigation exploite ces données pour calculer les commandes de déplacement et les publie sur le topic /cmd_vel.

Le nœud robot_controller, lorsqu'il est actif, s'abonne au topic /cmd_vel et pilote les actionneurs physiques avec les interfaces I2C et GPIO du Raspberry Pi. Les broches GPIO 17, 18, 27 et 22 contrôlent le driver moteur L298N pour les moteurs à courant continu, tandis que le module PCA9685 (SDA/SCL) gère le servomoteur via son canal 0.

4. Développement – état d'avancement

4.1 Présentation des composants développés



Capteur Adafruit IMU 9DoF FXOS8700 + FXAS21002

```
util@DroneCible:~$ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- 1f
20:  -- 21 -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- --
40: 40  -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- --
```

Capture d'écran des périphériques connectés en I2C à la Raspberry

Le capteur NXP Precision 9DoF Breakout est un module IMU (Inertial Measurement Unit) branché en I2C sur la Raspberry sur les périphériques 0x1f et 0x21, il est utilisé pour mesurer les mouvements et l'orientation d'un système. Il combine plusieurs capteurs permettant d'obtenir des informations précises sur l'accélération, la rotation et le champ magnétique.

Ce module intègre deux composants principaux :

- FXOS8700 (0x1f) : un capteur combinant un accéléromètre 3 axes et un magnétomètre 3 axes.
- FXAS21002 (0x21) : un gyroscope 3 axes.

Grâce aux capteurs, le module fournit 9 degrés de liberté (9DoF), ce qui permet de mesurer les mouvements dans l'espace selon les axes X, Y et Z. Il peut ainsi déterminer l'orientation et les déplacements d'un objet.



Capteur GPS BU-353

(mettre une trame)

Le GPS BU-353 se connecte en USB à la Raspberry. Il transmet les données de position sous forme de trames NMEA, qui contiennent différentes informations comme la latitude, la longitude, l'altitude, la vitesse et l'heure GPS.

Ce module est reconnu pour sa bonne sensibilité de réception et sa capacité à capter plusieurs satellites qui améliore la précision de localisation. Grâce à ces caractéristiques, il est particulièrement adapté pour des applications de géolocalisation et de navigation.

Dans le projet du drone cible, ce capteur est utilisé pour récupérer les coordonnées GPS du robot et permettre au système de navigation de suivre un itinéraire défini par des points de passage (waypoints).

4.2 Extrait de code commentés et expliqués

Dans le cadre du mode autonome du drone cible, deux nœuds principaux ont été développés : le nœud `imu_node` qui gère la lecture et la publication des données des capteurs inertiels, le code complet se trouve en [annexe](#), et le nœud `node_consigne` qui assure la gestion des

waypoints et le suivi de la mission, le code complet se trouve en [annexe](#). Les extraits de code présentés ci-dessous illustrent les parties les plus importantes de ces deux nœuds.

4.2.1 Nœud imu_node

Le nœud `imu_node` a pour rôle de lire les données des capteurs IMU (FXOS8700 et FXAS21002) via le bus I2C du Raspberry Pi, et de les publier sur le topic ROS2 `/imu/data` sous forme de messages de type `Imu`.

4.2.1.1 Initialisation du bus I2C et des capteurs

```
# I2C
i2c = busio.I2C(board.SCL, board.SDA)

# Capteurs
self.accel_mag = adafruit_fxos8700.FXOS8700(i2c)
self.gyro = adafruit_fxas21002c.FXAS21002C(i2c)

# Timer 100 Hz
self.timer = self.create_timer(0.01, self.publish_imu)
```

Les deux capteurs sont initialisés sur le même bus I2C grâce aux bibliothèques Adafruit. Le timer à 100 Hz garantit une fréquence de publication élevée et régulière, ce qui est essentiel pour que le nœud Robot Localization dispose de données suffisamment précises pour estimer la position et l'orientation du drone en temps réel.

4.2.1.2 Calcul du cap et publication des covariances

```
# Calcul du cap (yaw) à partir du magnétomètre
mx, my, mz = self.accel_mag.magnetometer
yaw = math.atan2(my, mx)

# Conversion en quaternion pour le message ROS2
roll = 0.0
pitch = 0.0
qx, qy, qz, qw = euler_to_quaternion(roll, pitch, yaw)
msg.orientation.x = qx
msg.orientation.y = qy
msg.orientation.z = qz
msg.orientation.w = qw

# Covariances pour le filtre de Kalman étendu (EKF)
msg.orientation_covariance[0] = 0.05
msg.orientation_covariance[4] = 0.05
msg.orientation_covariance[8] = 0.1
msg.angular_velocity_covariance[0] = 0.02
msg.angular_velocity_covariance[4] = 0.02
msg.angular_velocity_covariance[8] = 0.02
msg.linear_acceleration_covariance[0] = 0.1
msg.linear_acceleration_covariance[4] = 0.1
msg.linear_acceleration_covariance[8] = 0.1
```

Le cap du drone est calculé grâce à la fonction `atan2` appliquée aux composantes X et Y du magnétomètre, ce qui donne l'orientation du drone par rapport au nord magnétique. Le message ROS2 de type `Imu` exprimant l'orientation en quaternion, une conversion mathématique est appliquée. Dans notre cas, seul le yaw est calculé à partir du magnétomètre, le roll et le pitch étant fixés à 0 car le drone évolue sur un plan horizontal.

Les covariances représentent les niveaux d'incertitude associés à chaque mesure. Elles sont transmises au nœud `Robot Localization` pour alimenter son filtre de Kalman étendu (EKF). Plus la valeur est faible, plus le filtre accorde de confiance à la mesure correspondante.

4.2.2 Nœud `node_consigne`

Le nœud `node_consigne` a pour rôle de charger les waypoints définis dans un fichier GeoJSON, de suivre la progression du drone le long de ces points de passage, et de publier le waypoint courant sur le topic `/consigne_waypoint`, le fichier waypoint est présenté dans l'[annexe](#).

4.2.2.1 Chargement des waypoints depuis le fichier GeoJSON

```
def load_waypoints(self, file):
    with open(file, 'r') as f:
        data = json.load(f)

    waypoints = []

    for feature in data["features"]:
        if feature["geometry"]["type"] == "Point":
            lon, lat = feature["geometry"]["coordinates"]
            order = feature["properties"].get("order", 0)

            waypoints.append({
                "order": order,
                "lat": lat,
                "lon": lon
            })
    # Tri des waypoints par ordre de passage
    waypoints.sort(key=lambda x: x["order"])
    return waypoints
```

Cette fonction lit le fichier GeoJSON contenant les coordonnées GPS des points de passage de la mission. Chaque waypoint est défini par ses coordonnées en latitude et longitude, ainsi que par un attribut `order` qui détermine l'ordre de passage. Les waypoints sont ensuite triés automatiquement afin de garantir que le drone les parcourt dans le bon ordre, indépendamment de leur ordre d'écriture dans le fichier.

4.2.2.2 Calcul de la distance entre deux points GPS

```
def distance_gps(self, lat1, lon1, lat2, lon2):
    R = 6371000 # Rayon de la Terre en mètres
    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    delta_phi = math.radians(lat2 - lat1)
    delta_lambda = math.radians(lon2 - lon1)
    a = math.sin(delta_phi/2)**2 + \
        math.cos(phi1)*math.cos(phi2)*math.sin(delta_lambda/2)**2
    c = 2*math.atan2(math.sqrt(a), math.sqrt(1-a))
    return R*c
```

Cette fonction implémente la formule de Haversine, qui permet de calculer la distance en mètres entre deux coordonnées GPS en tenant compte de la courbure de la Terre. Elle est utilisée à chaque itération de la boucle principale pour déterminer si le drone est suffisamment proche du waypoint courant pour passer au point de passage suivant.

4.2.2.3 Boucle principale de gestion des waypoints

```
def main_loop(self):
    # Pas encore de GPS reçu
    if self.current_lat is None:
        self.get_logger().warn("Pas de GPS reçu (/fix)")
        return

    # Mission terminée
    if self.index_courant >= len(self.waypoints):
        self.get_logger().info("Mission terminée.")
        return

    # Calcul de la distance au waypoint courant
    wp = self.waypoints[self.index_courant]
    distance = self.distance_gps(
        self.current_lat, self.current_lon,
        wp["lat"], wp["lon"]
    )
    self.get_logger().info(
        f"WP {self.index_courant+1} | Distance: {distance:.2f} m"
    )

    # Si waypoint atteint → passer au suivant
    if distance <= self.tolerance:
        self.get_logger().info("Waypoint atteint ! Passage au suivant")
        self.index_courant += 1
        if self.index_courant < len(self.waypoints):
            self.publish_waypoint()
    else:
        # Publie uniquement le waypoint courant
        self.publish_waypoint()
```

Cette boucle principale s'exécute toutes les secondes. Elle gère trois cas distincts. Dans un premier temps, elle vérifie que des données GPS sont disponibles avant de commencer la navigation. Ensuite, elle contrôle si tous les waypoints ont été atteints pour signaler la fin de la mission. Enfin, elle calcule en permanence la distance entre la position actuelle du drone et le waypoint courant. Si cette distance est inférieure à la tolérance définie à 7 mètres, le nœud considère le waypoint comme atteint et publie automatiquement le suivant sur le topic

/consigne_waypoint. Dans le cas contraire, il continue de publier le waypoint courant jusqu'à ce que le drone l'atteigne.

4.3 Tableau ou cahier de tests unitaires

Les tests unitaires présentés ci-dessous ont pour objectif de valider le bon fonctionnement des différents composants développés dans le cadre du mode autonome. Chaque test vérifie une fonctionnalité précise d'un nœud de manière indépendante.

4.3.1 Test du noeud imu_node

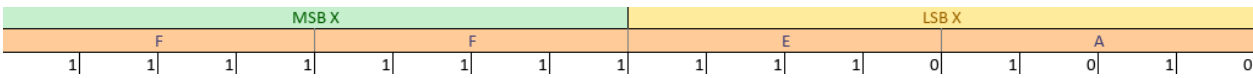
N°	Nœud	Description du test	Commande / Méthode	Résultat attendu	Résultat obtenu
T01	imu_node	Détection des capteurs sur le bus I2C	i2cdetect -y 1	Adresses 0x1F, 0x21 et 0x40 visibles	Validé
T02	imu_node	Démarrage du nœud sans erreur	ros2 run imu_node imu_data	Affichage du message "IMU 9DOF Node démarré"	Validé
T03	imu_node	Publication des données sur le topic /imu/data	ros2 topic echo /imu/data	Affichage des données d'accélération, gyroscope et orientation	Validé
T04	imu_node	Vérification de la fréquence de publication	ros2 topic hz /imu/data	Fréquence d'environ 100 Hz	Validé
T05	imu_node	Cohérence des données du magnétomètre	ros2 topic echo /imu/data	Valeur de yaw cohérente avec l'orientation physique du drone	Validé

4.3.2 Test de fonctionnement du capteur gyroscope FXAS21002 de l'IMU

Dans ce test étapes par étapes, je vais expliquer comment lire et décoder les trames I2C d'un capteur gyroscope FXAS21002 avec un PicoScope, la documentation technique officielle du capteur ici : <https://www.nxp.com/docs/en/data-sheet/FXAS21002.pdf> .

4.3.2.1 Ce que le capteur envoie

Voir en [annexe](#) la capture d'écran d'une trame du capteur gyroscope FXAS21002 au PicoScope.



Valeur brute sur 2 octets

Le FXAS21002C envoie la valeur brute sur 2 octets via I2C. Deux registres séparés, big-endian : le MSB (Most Significant Byte) arrive en premier.

4.3.2.2 Assembler les 2 octets en 16 bits

On colle les 2 octets bout à bout pour former un seul nombre de 16 bits. Le bit le plus a gauche indique le signe.

Bit 15 = 1 Donc c'est un nombre négatif.

0xFFEA = 0b1111111111101010 = 65514 en décimal non-signé.

4.3.2.3 Convertir en nombre signé

Pour trouver la valeur négative, on soustrait par 65536 (= 2¹⁶).

65514 - 65536 = -22

-22 est la valeur brute en LSB que le capteur a vraiment mesuré.

4.3.2.4 Convertir en °/s

Le capteur est configuré en ±2000 °/s. La sensibilité est de 62.5 mdps/LSB.

-22 x 62.5 mdps = -1375 mdps

-1375 mdps / 1000 = -1.375°/s

-1.375°/s est ce que mon programme doit afficher.

```
Accel: X=0.33 Y=-2.09 Z=39.83 m/s²
Mag:   X=5 Y=3 Z=-97 | Heading=31.0°
Gyro:  X=-1.38 Y=0.38 Z=0.06 °/s
```

Capture d'écran des valeurs du capteur de l'IMU correspondant à la trame du PicoScope

On peut bien voir que la valeur Gyro en X correspond bien au résultat obtenu.

4.3.3 Test du noeud node_consigne

N°	Noeud	Description du test	Commande / Méthode	Résultat attendu	Résultat obtenu
----	-------	---------------------	--------------------	------------------	-----------------

T06	node_consigne	Chargement correct du fichier GeoJSON	Lancement du nœud avec un fichier valide	Affichage du nombre de waypoints chargés dans les logs	Validé
T07	node_consigne	Réception des données GPS sur /fix	ros2 topic echo /fix	Affichage des coordonnées latitude et longitude	Validé
T08	node_consigne	Publication du waypoint courant	ros2 topic echo /consigne_waypoint	Affichage des coordonnées du premier waypoint	Validé
T09	node_consigne	Passage au waypoint suivant	Simuler une position proche du waypoint courant	Le nœud passe automatiquement au waypoint suivant	Validé
T10	node_consigne	Détection de fin de mission	Atteindre le dernier waypoint	Affichage du message "Mission terminée" dans les logs	Validé

4.4 Gestion des versions

Dans le cadre de mes développements, j'ai utilisé Git et le dépôt partagé Drone_cible_2026_repo pour versionner et centraliser mes contributions. Mes ajouts se retrouvent principalement dans les dossiers dédiés aux nœuds du mode autonome, notamment imu_node, node_consigne, ainsi que dans le dossier de documentation associé. La page du GitHub est présentée dans l'[annexe](#).

5. Exploitation réseau et matériels

5.1 Présentation des équipements

Dans le cadre du mode autonome du drone cible, plusieurs équipements matériels sont utilisés afin d'assurer la navigation autonome du drone et la communication entre les différents composants du système.

La Raspberry Pi 5 constitue l'ordinateur embarqué du drone. Elle exécute l'ensemble des nœuds nécessaires au fonctionnement du mode autonome, au sein d'un conteneur Docker utilisant ROS2-Jazzy.

La localisation géographique du drone est assurée par un module GPS connecté à la Raspberry Pi via une liaison USB/UART. Il transmet en continu les coordonnées de position du drone sous forme de trames au format NMEA, qui sont ensuite interprétées par le driver NMEA NavSat Driver.

L'orientation et les mouvements du drone sont mesurés par une IMU 9 degrés de liberté, composée de deux capteurs NXP connectés via le bus I2C. Le capteur FXOS8700 assure la mesure de l'accélération linéaire et du champ magnétique, tandis que le capteur FXAS21002 mesure la vitesse angulaire sur les trois axes.

Le pilotage des moteurs à courant continu est réalisé via un driver L298N, contrôlé directement par les broches GPIO du Raspberry Pi. Le contrôle du servomoteur est quant à lui assuré par un module PCA9685, un contrôleur PWM 16 canaux communiquant avec le Raspberry Pi via le bus I2C.

5.2 Justification des choix matériels

Le choix des équipements utilisés dans ce projet repose sur plusieurs critères, notamment la compatibilité avec le Raspberry Pi, la disponibilité des bibliothèques Python et l'adaptation aux besoins du système.

La Raspberry Pi 5 a été choisie pour sa capacité de calcul élevée, sa mémoire vive suffisante et la possibilité d'exécuter une distribution Linux, ce qui permet de faire fonctionner ROS2-Jazzy et Docker dans de bonnes conditions.

Le module GPS avec communication USB/UART a été retenu pour sa simplicité d'intégration. Le format NMEA qu'il utilise est un standard largement répandu dans le domaine de la navigation, et il est nativement supporté par le driver ROS2 NMEA NavSat Driver, ce qui facilite son intégration dans l'architecture du système.

Les capteurs FXOS8700 et FXAS21002 ont été choisis car ils sont directement compatibles avec le bus I2C du Raspberry Pi et disposent de bibliothèques Python officielles développées et maintenues par Adafruit. Leur combinaison permet d'obtenir une IMU 9DOF complète, offrant toutes les données nécessaires à l'estimation précise de l'orientation du drone.

Le driver L298N a été retenu pour sa capacité à piloter deux moteurs à courant continu de façon indépendante, tout en étant directement pilotable via les broches GPIO du Raspberry Pi sans nécessiter de composant supplémentaire.

Le module PCA9685 a été choisi pour générer le signal PWM destiné au servomoteur car le Raspberry Pi ne dispose que d'un nombre limité de sorties PWM matérielles. Ce module permet de pallier cette limitation tout en offrant une grande flexibilité pour de futures extensions du système.

5.3 Protocoles et formats de données échangés

La communication entre les différents équipements du drone cible repose sur plusieurs protocoles distincts, chacun adapté aux contraintes de l'équipement qu'il relie.

5.3.1 Protocole I2C

Le bus I2C est utilisé pour la communication entre le Raspberry Pi et les capteurs IMU ainsi qu'avec le module PCA9685. Il s'agit d'un protocole série synchrone fonctionnant sur deux fils : SDA (données) et SCL (horloge). Le Raspberry Pi joue le rôle de maître et interroge chaque périphérique via son adresse unique sur le bus I2C numéro 1, accessible via `/dev/i2c-1`.

Périphérique	Adresse I2C
FXOS8700 (accél. + magnéto.)	0x1F
FXAS21002 (gyroscope)	0x21
PCA9685 (contrôleur PWM)	0x40

5.3.2 Protocole UART / USB

Le module GPS communique avec le Raspberry Pi via une liaison série UART encapsulée dans une connexion USB. Les données transmises sont des trames au format NMEA. La trame \$GPGGA est la plus utilisée car elle contient les informations de position en latitude, longitude et altitude. Ces trames sont transmises à une fréquence d'une fois par seconde à un débit de 4800 bauds.

Exemple de trame NMEA \$GPGGA :

```
$GPGGA,133322.601,4824.5084,N,00428.8435,W,1,04,3.3,66.7,M,52.0,M,,0000*78
```

Capture d'une trame NMEA du GPS

5.3.3 Topics ROS2

Les données échangées entre les différents nœuds du système transitent via des topics ROS2. Le tableau ci-dessous récapitule les principaux topics utilisés dans le cadre du mode autonome :

Topic	Type de message	Émetteur	Récepteur
/imu/data	sensor_msgs/IMU	imu_node	Robot Localization
/fix	sensor_msgs/NavSatFix	NMEA NavSat Driver	Robot Localization
/odom_filter	nav_msgs/Odometry	Robot Localization	node_navigation
/consigne_waypoint	geometry_msgs/Point	node_consigne	node_navigation

5.3.4 GPIO et PWM

Le driver L298N est piloté via quatre broches GPIO du Raspberry Pi (17, 18, 27 et 22) à l'aide de signaux numériques et PWM pour contrôler respectivement le sens de rotation et la vitesse des moteurs. Le module PCA9685 génère un signal PWM à 50 Hz sur son canal 0 pour contrôler le servomoteur, ce qui correspond au standard utilisé par la majorité des servomoteurs RC.

5.4 Résultats d'outils d'analyse réseau

5.4.1 Détection des périphériques I2C

La commande `i2cdetect -y 1` permet de scanner le bus I2C du Raspberry Pi et de vérifier que tous les périphériques sont correctement connectés et accessibles.

```
util@DroneCible:~$ i2cdetect -y 1
      0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- 1f
20: -- 21 -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- --
40: 40 -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- --
```

Capture d'écran des périphériques connectés en I2C à la Raspberry

Cette capture confirme la présence des trois périphériques attendus : le FXOS8700 à l'adresse 0x1F, le FXAS21002 à l'adresse 0x21 et le PCA9685 à l'adresse 0x40.

5.4.2 Test de connectivité réseau

La connectivité entre le poste de développement et le Raspberry Pi est vérifiée à l'aide de la commande `ping`, qui permet de s'assurer que le Raspberry Pi est bien accessible sur le réseau local avant de démarrer une session SSH.

```
gjaumoui@x117-10:~$ ping 172.25.28.12
PING 172.25.28.12 (172.25.28.12) 56(84) bytes of data.
64 bytes from 172.25.28.12: icmp_seq=1 ttl=64 time=4.48 ms
64 bytes from 172.25.28.12: icmp_seq=2 ttl=64 time=5.00 ms
64 bytes from 172.25.28.12: icmp_seq=3 ttl=64 time=1.67 ms
64 bytes from 172.25.28.12: icmp_seq=4 ttl=64 time=5.10 ms
```

Capture d'écran de la commande ping avec l'adresse ip de la Raspberry

5.5 Gestion des versions

Le tableau ci-dessous récapitule les versions des différents matériels et logiciels utilisés dans le cadre du mode autonome du drone cible.

Élément	Version
Raspberry Pi	5
ROS2	Jazzy
Python	3.12.3
Docker	29.3.1
ros-jazzy-nmea-navsat-driver	Jazzy
ros-jazzy-robot-localization	Jazzy

5.6 Cybersécurité

Plusieurs mesures ont été mises en place afin d'assurer la sécurité du système embarqué du drone cible.

L'accès à la Raspberry Pi 5 se fait à distance via le protocole SSH, qui chiffre l'ensemble des communications entre le poste de développement et le système embarqué. L'accès est protégé par un mot de passe afin de limiter les connexions non autorisées au système.

Le réseau Wi-Fi utilisé pour les communications locales lors des phases de développement et de test est sécurisé avec le protocole WPA2, garantissant un chiffrement des échanges sur le réseau sans fil.

L'utilisation de Docker sur la Raspberry Pi permet d'isoler l'environnement du mode autonome du reste du système d'exploitation. Cette isolation limite les interactions directes entre les différentes applications et améliore la sécurité globale du système en réduisant la surface d'attaque.

Enfin, le système d'exploitation Raspberry Pi OS ainsi que les bibliothèques Python utilisées sont maintenus à jour régulièrement via les commandes `apt update` et `apt upgrade`, afin de corriger les éventuelles failles de sécurité connues.

6. Bilan personnel

6.1 Ce que J'ai réalisé

Au cours de ce projet, j'ai développé l'ensemble des composants nécessaires au mode autonome du drone cible. J'ai mis en place le nœud `imu_node`, permettant la lecture et la publication des données des capteurs FXOS8700 et FXAS21002 via le bus I2C. J'ai également développé le nœud `node_consigne`, qui assure le chargement des waypoints depuis un fichier GeoJSON et la gestion de la progression de la mission. J'ai intégré le driver NMEA NavSat Driver pour la réception des données GPS, ainsi que le nœud Robot Localization pour la fusion des données IMU et GPS. L'ensemble de ces composants a été déployé dans un conteneur

Docker sur le Raspberry Pi 5. La seule partie restante à finaliser est le nœud `node_navigation`, qui constituera ma prochaine étape de développement.

6.2 Difficultés rencontrées et solutions apportées

La principale difficulté rencontrée au cours de ce projet a été d'obtenir des données cohérentes et exploitables provenant du capteur IMU. En effet, les premières lectures des capteurs FXOS8700 et FXAS21002 retournaient des valeurs instables ou incohérentes, ce qui rendait impossible une estimation fiable de l'orientation du drone. Pour résoudre ce problème, j'ai analysé en détail le fonctionnement des bibliothèques Adafruit et vérifié la configuration du bus I2C. J'ai également travaillé sur le paramétrage des covariances transmises au filtre de Kalman étendu du nœud Robot Localization, afin d'ajuster le niveau de confiance accordé à chaque mesure et d'obtenir une fusion de données plus stable et précise.

6.3 Ce que j'ai appris

Ce projet m'a permis d'acquérir de nombreuses compétences nouvelles. J'ai découvert et appris à utiliser ROS2 dans un contexte réel, en développant des nœuds Python communiquant entre eux via des topics, ce qui m'a permis de comprendre concrètement l'architecture modulaire de cette plateforme robotique. J'ai également appris à utiliser Docker pour conteneuriser un environnement logiciel complet, ce qui m'a sensibilisé à l'importance de l'isolation et de la reproductibilité des environnements de développement. Plus globalement, ce projet m'a permis de mettre en pratique des connaissances en électronique embarquée, en communication I2C et en navigation autonome, dans le cadre d'un projet concret et collaboratif.

Ce projet m'a également permis de prendre conscience de l'importance d'une bonne gestion des versions avec Git. En effet, ayant commis trop peu de commits réguliers tout au long du développement. Pour les projets futurs, je veillerai à effectuer des commits plus fréquents, afin de mieux documenter la progression de mon travail.

Annexes

[1] Versionnement git

guivarchVauban / Drone_cible_2026_repo

<> Code

Issues 13

Pull requests

Agents

Actions

Projects

Security and quality

Insights

Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

Files

main

Go to file

code

VM_Ubuntu

raspberrypi

ProjetoDroneJazzy

ProjetoDrone

m_auto

ros_ws

Waypoints

src

gps_simulator

imu_node

node_consigne

gps.txt

Dockerfile

docker-compose.yml

m_manuel

m_safemode

m_verrou

Drone_cible_2026_repo / code / raspberrypi / ProjetoDroneJazzy / m_auto / ros_ws / src

root and root

ajout du code v2 depuis la Raspberry Pi

b8eff1d · last week

History

Name	Last commit message	Last commit date
..		
gps_simulator	ajout du code v2 depuis la Raspberry Pi	last week
imu_node	ajout du code v2 depuis la Raspberry Pi	last week
node_consigne	ajout du code v2 depuis la Raspberry Pi	last week
gps.txt	ajout du code v2 depuis la Raspberry Pi	last week

Rapport individuel

21/34

[\[2\] Code complets du noeud imu_node : imu_data.py](#)

```
import rclpy

from rclpy.node import Node

from sensor_msgs.msg import Imu

import math

import board

import busio

import adafruit_fxos8700

import adafruit_fxas21002c


def euler_to_quaternion(roll, pitch, yaw):

    qx = math.sin(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) -
math.cos(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)

    qy = math.cos(roll/2) * math.sin(pitch/2) * math.cos(yaw/2) +
math.sin(roll/2) * math.cos(pitch/2) * math.sin(yaw/2)

    qz = math.cos(roll/2) * math.cos(pitch/2) * math.sin(yaw/2) -
math.sin(roll/2) * math.sin(pitch/2) * math.cos(yaw/2)

    qw = math.cos(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) +
math.sin(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)

    return qx, qy, qz, qw


class ImuNode(Node):

    def __init__(self):

        super().__init__('imu_node')
```



```
self.publisher_ = self.create_publisher(Imu, '/imu/data', 10)

# I2C

i2c = busio.I2C(board.SCL, board.SDA)

# Capteurs

self.accel_mag = adafruit_fxos8700.FXOS8700(i2c)

self.gyro = adafruit_fxas21002c.FXAS21002C(i2c)

# Timer 100 Hz

self.timer = self.create_timer(0.01, self.publish_imu)

self.get_logger().info("IMU 9DOF Node démarré")

def publish_imu(self):

    msg = Imu()

    msg.header.stamp = self.get_clock().now().to_msg()

    msg.header.frame_id = "base_link"

    # =====

    # ACCELERATION

    # =====

    ax, ay, az = self.accel_mag.accelerometer

    msg.linear_acceleration.x = ax

    msg.linear_acceleration.y = ay
```

```
msg.linear_acceleration.z = az

# =====

# GYROSCOPE (rad/s)

# =====

gx, gy, gz = self.gyro.gyroscope

msg.angular_velocity.x = gx
msg.angular_velocity.y = gy
msg.angular_velocity.z = gz

# =====

# MAGNETOMETRE → YAW

# =====

mx, my, mz = self.accel_mag.magnetometer

yaw = math.atan2(my, mx)

# =====

# ORIENTATION

# =====

roll = 0.0
pitch = 0.0

qx, qy, qz, qw = euler_to_quaternion(roll, pitch, yaw)

msg.orientation.x = qx
msg.orientation.y = qy
```

```
msg.orientation.z = qz

msg.orientation.w = qw


# =====

# COVARIANCES (IMPORTANT EKF)

# =====

msg.orientation_covariance[0] = 0.05

msg.orientation_covariance[4] = 0.05

msg.orientation_covariance[8] = 0.1


msg.angular_velocity_covariance[0] = 0.02

msg.angular_velocity_covariance[4] = 0.02

msg.angular_velocity_covariance[8] = 0.02


msg.linear_acceleration_covariance[0] = 0.1

msg.linear_acceleration_covariance[4] = 0.1

msg.linear_acceleration_covariance[8] = 0.1


self.publisher_.publish(msg)


def main():

    rclpy.init()

    node = ImuNode()

    rclpy.spin(node)

    node.destroy_node()

    rclpy.shutdown()
```

```
if __name__ == '__main__':  
    main()
```

[\[3\] Code complet du noeud node_consigne : node_consigne.py](#)

```
import rclpy  
  
from rclpy.node import Node  
  
from geometry_msgs.msg import Point  
from sensor_msgs.msg import NavSatFix  
  
import json  
import math  
  
class NoeudConsigne(Node):  
  
    def __init__(self):  
        super().__init__('node_consigne')  
  
        # Paramètres  
  
        self.declare_parameter("geojson_file",  
                                "/root/ros_ws/Waypoints/waypoints.json")  
  
        self.declare_parameter("tolerance", 7.0) # entre 5 et 10m  
  
        geojson_file = self.get_parameter("geojson_file").value  
        self.tolerance = self.get_parameter("tolerance").value
```

```
# Chargement waypoints

self.waypoints = self.load_waypoints(geojson_file)

self.index_courant = 0


# Position GPS actuelle

self.current_lat = None

self.current_lon = None


# Subscriber GPS (/fix)

self.subscription = self.create_subscription(

    NavSatFix,

    "/fix",

    self.gps_callback,

    10

)


# Publisher waypoint

self.publisher_ = self.create_publisher(Point,

"/consigne_waypoint", 10)


# Timer

self.timer = self.create_timer(1.0, self.main_loop)


self.get_logger().info(f"{len(self.waypoints)} waypoints chargés.")

self.get_logger().info("Node Consigne ROS2 démarré.")


def load_waypoints(self, file):

    with open(file, 'r') as f:
```

```
data = json.load(f)

waypoints = []

for feature in data["features"]:

    if feature["geometry"]["type"] == "Point":

        lon, lat = feature["geometry"]["coordinates"]

        order = feature["properties"].get("order", 0)

        waypoints.append({

            "order": order,

            "lat": lat,

            "lon": lon

        })

waypoints.sort(key=lambda x: x["order"])

return waypoints


def gps_callback(self, msg):

    self.current_lat = msg.latitude

    self.current_lon = msg.longitude


def distance_gps(self, lat1, lon1, lat2, lon2):

    R = 6371000

    phi1 = math.radians(lat1)

    phi2 = math.radians(lat2)
```

```
delta_phi = math.radians(lat2 - lat1)

delta_lambda = math.radians(lon2 - lon1)

a = math.sin(delta_phi/2)**2 + \
    math.cos(phi1)*math.cos(phi2)*math.sin(delta_lambda/2)**2

c = 2*math.atan2(math.sqrt(a), math.sqrt(1-a))

return R*c

def publish_waypoint(self):

    wp = self.waypoints[self.index_courant]

    msg = Point()

    msg.x = wp["lat"]

    msg.y = wp["lon"]

    msg.z = 0.0

    self.publisher_.publish(msg)

    self.get_logger().info(
        f"Publication WP {self.index_courant+1} → lat: {wp['lat']} lon: {wp['lon']}"
    )

def main_loop(self):

    # Pas encore de GPS reçu

    if self.current_lat is None:
```

```
        self.get_logger().warn("Pas de GPS reçu (/fix)")

        return

# Mission terminée

if self.index_courant >= len(self.waypoints):

    self.get_logger().info("Mission terminée.")

    return

wp = self.waypoints[self.index_courant]

distance = self.distance_gps(

    self.current_lat,

    self.current_lon,

    wp["lat"],

    wp["lon"]

)

self.get_logger().info(

    f"WP {self.index_courant+1} | Distance: {distance:.2f} m"

)

# Si waypoint atteint → passer au suivant

if distance <= self.tolerance:

    self.get_logger().info("Waypoint atteint ! Passage au suivant.")

    self.index_courant += 1

    if self.index_courant < len(self.waypoints):
```



```
        self.publish_waypoint()

    else:

        # Publie uniquement le waypoint courant

        self.publish_waypoint()

# === MAIN ROS2 ===
def main(args=None):

    rclpy.init(args=args)

    node = NoeudConsigne()

    rclpy.spin(node)

    node.destroy_node()

    rclpy.shutdown()

if __name__ == '__main__':

    main()
```

[\[4\] Fichier waypoints en GEOJson : waypoints.json](#)

```
{

  "type": "FeatureCollection",

  "features": [

    {

      "type": "Feature",

      "properties": {

        "order": 1,
```

```
        "name": "Waypoint 1"

    },

    "geometry": {

        "type": "Point",

        "coordinates": [

            -4.478037,

            48.409209

        ]

    }

},

{

    "type": "Feature",

    "properties": {

        "order": 2,

        "name": "Waypoint 2"

    },

    "geometry": {

        "type": "Point",

        "coordinates": [

            -4.477341,

            48.409359

        ]

    }

},

{

    "type": "Feature",

    "properties": {

        "order": 3,
```

```
        "name": "Waypoint 3"

    },

    "geometry": {

        "type": "Point",

        "coordinates": [

            -4.478175,

            48.409466

        ]

    }

},

{

    "type": "Feature",

    "properties": {

        "order": 4,

        "name": "Waypoint 4"

    },

    "geometry": {

        "type": "Point",

        "coordinates": [

            -4.47747,

            48.409622

        ]

    }

}

]

}
```

[5] Capture d'écran d'une trame du capteur FXAS21002 au PicoScope

